Week 3 Structured Notes: Resizing and 1D Array Address Formulas

Topic Overview

Week 3 connects two important ideas:

1. **Manual resizing of heap arrays**
2. **1D array address formulas**

In Week 1, you learned how to create and use arrays.

In Week 2, you learned about stack memory, heap memory, pointers, `malloc`, `free`, `new[]`, `delete[]`, memory leaks, and copying arrays.

Week 3 now explains what happens when an array needs to become bigger, and how the computer calculates the memory address of any element in a one-dimensional array.

This week mainly covers:

- Why arrays cannot grow directly
- Contiguous memory
- Manual heap array resizing
- Helper pointer `Q`
- Copying old values into a larger array
- Releasing old memory safely
- Avoiding memory leaks during resizing
- Base address
- Word size
- Index
- 0-indexed address formula
- 1-indexed address formula
- Why C and C++ use 0-based indexing
- Full C practice program
- Common mistakes
- Practice tasks

---

1. What You Should Know Before Week 3

Before studying Week 3, you should already understand these Week 1 and Week 2 ideas.

From Week 1

You should know that:

- A variable stores one value.
- An array stores many values under one name.
- Array indexing in C and C++ starts from `0`.
- A 5-element array has valid indexes `0` to `4`.
- A loop is used to visit every array element.
- Array elements are stored side by side in memory.
- Each element has a memory address.
- Accessing one array element is fast because the computer can calculate its location directly.

Example:

```
int A[5] = {2, 4, 6, 8, 10};
```

Logical view:

```
Index:  0   1   2   3   4
Value:  2   4   6   8   10
```

Memory view, assuming one `int` takes 4 bytes:

```
A[0] address = 200
A[1] address = 204
A[2] address = 208
A[3] address = 212
A[4] address = 216
```

The address increases by `4` each time because each integer uses 4 bytes.

From Week 2

You should know that:

- Stack arrays are fixed-size arrays.
- Heap arrays are created dynamically while the program is running.
- A pointer stores a memory address.
- A heap array needs a pointer because the actual array is stored in heap memory.
- In C, `malloc` creates heap memory.
- In C, `free` releases heap memory.

- In C++, `new[]` creates a heap array.
- In C++, `delete[]` releases a heap array.
- If heap memory is not released, a memory leak happens.
- Copying arrays is important before learning resizing.

Example from Week 2:

```
P -> [1] [2] [3] [4] [5]
Q -> [ ] [ ] [ ] [ ] [ ]
```

After copying:

```
P -> [1] [2] [3] [4] [5]
Q -> [1] [2] [3] [4] [5]
```

Week 3 uses the same idea, but instead of copying into another array of the same size, we copy into a **larger** array.

---

2. Main Idea of Week 3

Week 3 has two main goals.

Goal 1: Understand resizing

A normal array cannot grow directly.

So, if we need a bigger heap array, we create a new larger array, copy the old values, free the old memory, and make the main pointer point to the new array.

Simple resizing process:

```
Create bigger array -> copy old values -> free old array -> redirect pointer
```

Goal 2: Understand 1D address formulas

The computer does not search for `A[i]` one element at a time.

Instead, it calculates the address using this formula:

```
Address of A[i] = L0 + (i * w)
```

This formula is for 0-indexed arrays, like arrays in C and C++.

Part A: Manual Resizing of Heap Arrays

---

3. Why Arrays Cannot Grow Directly

An array needs **contiguous memory**.

Contiguous memory means memory locations are placed directly side by side.

Example:

```
P -> [1] [2] [3] [4] [5]
```

This means the array elements are stored together in one continuous block.

Now suppose we want to increase the array from size 5 to size 10.

We may want this:

```
P -> [1] [2] [3] [4] [5] [ ] [ ] [ ] [ ] [ ]
```

But this may not be possible.

The memory after the array may already be used by something else.

Example:

```
P -> [1] [2] [3] [4] [5] [Used by another variable]
```

Because of this, the array cannot simply stretch forward.

Simple beginner explanation

Imagine you booked 5 seats in a row:

```
[Seat 1] [Seat 2] [Seat 3] [Seat 4] [Seat 5]
```

Later, you want 5 more seats beside them.

But the next seat may already be booked by someone else.

So, you cannot expand the same row.

Instead, you must find a new row with 10 seats, move your people there, and release the old 5 seats.

Arrays work in a similar way.

---

4. The Correct Resizing Idea

To resize a heap array manually, follow these steps:

```
Step 1: Create the old array using pointer P
Step 2: Store values in P
Step 3: Create a bigger array using helper pointer Q
Step 4: Copy old values from P to Q
Step 5: Free old memory used by P
Step 6: Make P point to Q
Step 7: Set Q to NULL
```

Simple memory view:

```
Old array:
P -> [1] [2] [3] [4] [5]

New bigger array:
Q -> [ ] [ ] [ ] [ ] [ ] [ ] [ ] [ ] [ ] [ ]
```

After copying:

```
P -> [1] [2] [3] [4] [5]
Q -> [1] [2] [3] [4] [5] [ ] [ ] [ ] [ ] [ ]
```

After freeing old memory and redirecting:

```
P -> [1] [2] [3] [4] [5] [ ] [ ] [ ] [ ] [ ]
Q -> NULL
```

Now `P` points to the new bigger array.

---

5. Why We Need a Helper Pointer Q

The helper pointer `Q` is used to safely store the address of the new bigger array.

Suppose we do this directly:

```
P = (int*)malloc(10 * sizeof(int));
```

This is dangerous if `P` was already pointing to an old heap array.

Before:

```
P -> [1] [2] [3] [4] [5]
```

After assigning new memory directly to `P`:

```
P -> [ ] [ ] [ ] [ ] [ ] [ ] [ ] [ ] [ ] [ ]
```

Now the old array still exists in memory, but we lost its address.

That means we cannot free it anymore.

This causes a **memory leak**.

So we use `Q`:

```
P keeps old array address
Q gets new bigger array address
```

Then we copy from `P` to `Q`, free `P`, and finally make `P = Q`.

---

6. Manual Resizing Code in C

This program resizes a heap array from size 5 to size 10.

```c
#include <stdio.h>
#include <stdlib.h>

int main() {
    int *P;
    int *Q;

    // Step 1: Create old heap array of size 5
    P = (int*)malloc(5 * sizeof(int));

    if (P == NULL) {
        printf("Memory allocation failed.\n");
        return 1;
    }

    // Step 2: Store values in P
    for (int i = 0; i < 5; i++) {
        P[i] = i + 1;
    }

    printf("Old array:\n");
    for (int i = 0; i < 5; i++) {
        printf("%d ", P[i]);
    }

    // Step 3: Create bigger heap array of size 10
    Q = (int*)malloc(10 * sizeof(int));

    if (Q == NULL) {
        printf("\nMemory allocation failed.\n");
        free(P);
        return 1;
    }

    // Step 4: Copy old values from P to Q
    for (int i = 0; i < 5; i++) {
        Q[i] = P[i];
    }

    // Step 5: Free old memory
    free(P);
```

```
    // Step 6: Make P point to the new bigger array
    P = Q;

    // Step 7: Make Q point to nothing
    Q = NULL;

    printf("\n\nResized array:\n");
    for (int i = 0; i < 5; i++) {
        printf("%d ", P[i]);
    }

    // Step 8: Free final array
    free(P);
    P = NULL;

    return 0;
}
```

Expected output:

```
Old array:
1 2 3 4 5

Resized array:
1 2 3 4 5
```

Important note:

The array now has size 10, but only the first 5 values were copied.

The remaining 5 spaces exist, but they may contain unknown values unless we initialize them.

---

7. Initializing the New Extra Spaces

After resizing from 5 to 10, the new spaces should be given safe values.

Example:

```
for (int i = 5; i < 10; i++) {
    P[i] = 0;
}
```

Now the full resized array becomes:

```
[1] [2] [3] [4] [5] [0] [0] [0] [0] [0]
```

This is safer than leaving the new elements uninitialized.

---

8. Improved Resizing Program in C

This version initializes the new spaces to `0` and prints all 10 values.

```c
#include <stdio.h>
#include <stdlib.h>

int main() {
    int oldSize = 5;
    int newSize = 10;

    int *P = (int*)malloc(oldSize * sizeof(int));

    if (P == NULL) {
        printf("Memory allocation failed.\n");
        return 1;
    }

    for (int i = 0; i < oldSize; i++) {
        P[i] = i + 1;
    }

    printf("Old array:\n");
    for (int i = 0; i < oldSize; i++) {
        printf("%d ", P[i]);
    }

    int *Q = (int*)malloc(newSize * sizeof(int));

    if (Q == NULL) {
        printf("\nMemory allocation failed.\n");
        free(P);
        return 1;
    }

    for (int i = 0; i < oldSize; i++) {
        Q[i] = P[i];
```

```
    }

    for (int i = oldSize; i < newSize; i++) {
        Q[i] = 0;
    }

    free(P);
    P = Q;
    Q = NULL;

    printf("\n\nResized array:\n");
    for (int i = 0; i < newSize; i++) {
        printf("%d ", P[i]);
    }

    free(P);
    P = NULL;

    return 0;
}
```

Expected output:

```
Old array:
1 2 3 4 5

Resized array:
1 2 3 4 5 0 0 0 0 0
```

---

9. Resizing Step-by-Step Trace

Suppose the old array is:

```
P -> [1] [2] [3] [4] [5]
```

Step 1: Create new bigger array

```
P -> [1] [2] [3] [4] [5]
Q -> [ ] [ ] [ ] [ ] [ ] [ ] [ ] [ ] [ ] [ ]
```

Step 2: Copy values

```
P -> [1] [2] [3] [4] [5]
Q -> [1] [2] [3] [4] [5] [ ] [ ] [ ] [ ] [ ]
```

Step 3: Initialize extra spaces

```
P -> [1] [2] [3] [4] [5]
Q -> [1] [2] [3] [4] [5] [0] [0] [0] [0] [0]
```

Step 4: Free old P memory

```
Old P block is released.
Q still points to the new bigger block.
```

Step 5: Redirect P

```
P = Q;
```

Now:

```
P -> [1] [2] [3] [4] [5] [0] [0] [0] [0] [0]
Q -> same block
```

Step 6: Set Q to NULL

```
Q = NULL;
```

Final view:

```
P -> [1] [2] [3] [4] [5] [0] [0] [0] [0] [0]
Q -> NULL
```

---

10. Common Resizing Mistakes

Mistake 1: Forgetting to free old memory

Wrong:

```
P = Q;
```

Problem:

The old array is lost without being released.

That causes a memory leak.

Correct:

```
free(P);
P = Q;
Q = NULL;
```

Mistake 2: Freeing P too early

Wrong:

```
free(P);

for (int i = 0; i < 5; i++) {
    Q[i] = P[i];
}
```

Problem:

After `free(P)`, the old memory no longer belongs to the program.

So reading `P[i]` after `free(P)` is unsafe.

Correct:

```c
for (int i = 0; i < 5; i++) {
    Q[i] = P[i];
}

free(P);
```

Mistake 3: Not checking whether malloc worked

Wrong:

```c
int *Q = (int*)malloc(10 * sizeof(int));
Q[0] = 10;
```

Problem:

If `malloc` fails, `Q` will be `NULL`.

Using `Q[0]` when `Q` is `NULL` can crash the program.

Correct:

```c
int *Q = (int*)malloc(10 * sizeof(int));

if (Q == NULL) {
    printf("Memory allocation failed.\n");
    free(P);
    return 1;
}
```

Mistake 4: Forgetting that P and Q point to the same block after P = Q

After this:

```c
P = Q;
```

Both pointers point to the same memory block.

```
P -> [1] [2] [3] [4] [5] [0] [0] [0] [0] [0]
Q -> same block
```

To avoid confusion, do this:

```
Q = NULL;
```

Now only `P` is used as the main pointer.

Mistake 5: Accessing outside the new array

If the new size is 10, valid indexes are:

```
0, 1, 2, 3, 4, 5, 6, 7, 8, 9
```

Wrong:

```
P[10] = 100;
```

`P[10]` is outside the array.

Part B: 1D Array Address Formula

---

11. Why Address Formulas Matter

When you write:

```
A[3]
```

the computer does not search like this:

```
A[0] -> A[1] -> A[2] -> A[3]
```

Instead, it calculates the address directly.

That is why accessing one array element is fast.

The computer uses:

```
Base address + distance from the start
```

---

12. Important Terms for Address Calculation

1. Base Address

The base address is the starting address of the array.

Usually, it is the address of the first element.

```
Base address = address of A[0]
```

Example:

```
A[0] address = 200
```

So:

```
L0 = 200
```

`L0` means base address.

---

2. Index

The index is the position number of the element.

Example:

```
A[3]
```

Here:

```
i = 3
```

Important:

In C and C++, index `3` means the fourth element, because indexing starts from `0`.

---

3. Word Size

Word size means the number of bytes used by one array element.

It is usually written as:

```
w
```

Example:

If one `int` takes 4 bytes:

```
w = 4
```

Common examples:

```
char   -> usually 1 byte
int    -> often 4 bytes
double -> often 8 bytes
```

The exact size can depend on the system, so in C we often use `sizeof`.

Example:

```
sizeof(int)
```

This asks the compiler how many bytes an `int` uses on that system.

---

13. 0-Indexed Array Address Formula

C and C++ use 0-based indexing.

That means:

```
A[0] = first element
A[1] = second element
A[2] = third element
A[3] = fourth element
```

The formula is:

```
Address of A[i] = L0 + (i * w)
```

Where:

```
L0 = base address
i  = index
w  = word size
```

---

14. Example: 0-Indexed Address Calculation

Given:

```
Base address L0 = 200
Index i = 3
Word size w = 4
```

Find:

```
Address of A[3]
```

Formula:

```
Address of A[i] = L0 + (i * w)
```

Substitute values:

```
Address of A[3] = 200 + (3 * 4)
                = 200 + 12
                = 212
```

Answer:

```
Address of A[3] = 212
```

Memory view:

```
A[0] -> 200
A[1] -> 204
A[2] -> 208
A[3] -> 212
A[4] -> 216
```

---

15. Why A[3] Is the Fourth Element

This is a common beginner mistake.

In C and C++:

```
A[0] = 1st element
A[1] = 2nd element
A[2] = 3rd element
A[3] = 4th element
```

So `A[3]` is not the third element.

It is the element at index `3`, which is the fourth element.

Why?

Because counting starts from `0`.

For `A[3]`, the computer skips 3 elements:

```
Skip A[0], A[1], A[2]
```

If each element uses 4 bytes:

```
3 * 4 = 12 bytes
```

Then:

```
200 + 12 = 212
```

So `A[3]` is stored at address `212`.

---

16. Another 0-Indexed Example

Given:

```
L0 = 1000
w = 2
i = 4
```

Formula:

```
Address of A[i] = L0 + (i * w)
```

Substitute:

```
Address of A[4] = 1000 + (4 * 2)
                = 1000 + 8
                = 1008
```

Answer:

```
Address of A[4] = 1008
```

---

17. 1-Indexed Array Address Formula

Some languages or textbook examples use 1-based indexing.

That means:

```
A[1] = first element
A[2] = second element
A[3] = third element
```

For 1-indexed arrays, the formula is:

```
Address of A[i] = L0 + ((i - 1) * w)
```

Why do we use `i - 1`?

Because if `A[1]` is the first element, it should be at the base address.

Example:

```
Address of A[1] = L0 + ((1 - 1) * w)
                = L0 + (0 * w)
                = L0
```

So the first element is stored at the base address.

---

18. Example: 1-Indexed Address Calculation

Given:

```
Base address L0 = 500
Index i = 4
Word size w = 2
```

Formula:

```
Address of A[i] = L0 + ((i - 1) * w)
```

Substitute values:

```
Address of A[4] = 500 + ((4 - 1) * 2)
                = 500 + (3 * 2)
```

```
                = 500 + 6
                = 506
```

Answer:

```
Address of A[4] = 506
```

---

19. 0-Indexed vs 1-Indexed Comparison

Given:

```
L0 = 1000
w = 4
i = 3
```

Using 0-indexing

Formula:

```
Address of A[i] = L0 + (i * w)
```

Calculation:

```
Address of A[3] = 1000 + (3 * 4)
                = 1000 + 12
                = 1012
```

In 0-indexing:

```
A[3] = fourth element
```

Using 1-indexing

Formula:

```
Address of A[i] = L0 + ((i - 1) * w)
```

Calculation:

```
Address of A[3] = 1000 + ((3 - 1) * 4)
                = 1000 + (2 * 4)
                = 1000 + 8
                = 1008
```

In 1-indexing:

```
A[3] = third element
```

Main difference

The same written expression `A[3]` can mean different physical positions depending on whether the array is 0-indexed or 1-indexed.

In C and C++, use the 0-indexed formula.

---

20. Why C and C++ Use 0-Based Indexing

C and C++ use 0-based indexing because it fits naturally with memory calculation.

The formula is simple:

```
Address of A[i] = L0 + (i * w)
```

For `A[0]`:

```
Address of A[0] = L0 + (0 * w)
                = L0
```

So the first element is exactly at the base address.

For 1-indexing, the formula needs an extra subtraction:

```
Address of A[i] = L0 + ((i - 1) * w)
```

So 0-indexing avoids that extra step.

Simple memory aid:

```
0-indexing: offset = i
1-indexing: offset = i - 1
```

---

21. Address Calculation Table

Assume:

```
L0 = 200
w = 4
```

Using 0-indexing:

Element	Formula	Address
A[0]	200 + (0 * 4)	200
A[1]	200 + (1 * 4)	204
A[2]	200 + (2 * 4)	208
A[3]	200 + (3 * 4)	212
A[4]	200 + (4 * 4)	216

This table shows why the address increases by 4 each time.

---

22. Full Week 3 Practice Program in C

This program covers:

- Creating a heap array
- Printing values and addresses
- Resizing the heap array
- Initializing new spaces
- Printing the resized array
- Manually calculating a 1D array address
- Releasing memory safely

```c
#include <stdio.h>
#include <stdlib.h>

int main() {
    int oldSize = 5;
    int newSize = 10;

    int *P = (int*)malloc(oldSize * sizeof(int));

    if (P == NULL) {
        printf("Memory allocation failed.\n");
        return 1;
    }

    for (int i = 0; i < oldSize; i++) {
        P[i] = i + 1;
    }

    printf("Old array:\n");
    for (int i = 0; i < oldSize; i++) {
        printf("P[%d] = %d, Address = %p\n", i, P[i], (void*)&P[i]);
    }

    int *Q = (int*)malloc(newSize * sizeof(int));

    if (Q == NULL) {
        printf("Memory allocation failed.\n");
        free(P);
        return 1;
    }

    for (int i = 0; i < oldSize; i++) {
        Q[i] = P[i];
    }

    for (int i = oldSize; i < newSize; i++) {
        Q[i] = 0;
    }

    free(P);
    P = Q;
    Q = NULL;

    printf("\nResized array:\n");
    for (int i = 0; i < newSize; i++) {
        printf("P[%d] = %d, Address = %p\n", i, P[i], (void*)&P[i]);
    }
```

```
    int L0 = 200;
    int index = 3;
    int w = 4;

    int address = L0 + (index * w);

    printf("\nManual address calculation:\n");
    printf("Address of A[%d] = %d + (%d * %d)\n", index, L0, index, w);
    printf("Address of A[%d] = %d\n", index, address);

    free(P);
    P = NULL;

    return 0;
}
```

---

23. Explanation of the Practice Program

This part creates the old heap array

```
int *P = (int*)malloc(oldSize * sizeof(int));
```

This creates space for `oldSize` integers in heap memory.

If `oldSize = 5`, it creates space for 5 integers.

This part checks whether memory allocation worked

```
if (P == NULL) {
    printf("Memory allocation failed.\n");
    return 1;
}
```

If `malloc` fails, the program stops safely.

This part fills the old array

```
for (int i = 0; i < oldSize; i++) {
    P[i] = i + 1;
}
```

The values become:

```
P -> [1] [2] [3] [4] [5]
```

This part creates the bigger array

```
int *Q = (int*)malloc(newSize * sizeof(int));
```

If `newSize = 10`, it creates space for 10 integers.

This part copies old values

```
for (int i = 0; i < oldSize; i++) {
    Q[i] = P[i];
}
```

Now the first 5 values of `Q` are copied from `P`.

This part initializes the extra spaces

```
for (int i = oldSize; i < newSize; i++) {
    Q[i] = 0;
}
```

This sets the new spaces to `0`.

This part releases old memory

```
free(P);
```

The old 5-element block is returned to the system.

This part redirects P

```
P = Q;
Q = NULL;
```

Now `P` points to the new 10-element array.

`Q` is set to `NULL` because it is no longer needed.

This part calculates address manually

```
int address = L0 + (index * w);
```

For:

```
L0 = 200
index = 3
w = 4
```

The calculation is:

```
200 + (3 * 4) = 212
```

---

24. Week 3 Mental Model

Think of an array like a row of lockers.

```
[Locker 0] [Locker 1] [Locker 2] [Locker 3] [Locker 4]
```

Each locker has the same size.

If each locker takes 4 bytes and the first locker starts at address 200:

```
Locker 0 -> 200
Locker 1 -> 204
Locker 2 -> 208
Locker 3 -> 212
Locker 4 -> 216
```

To find locker 3:

```
Start address + number of lockers skipped * locker size
200 + 3 * 4
212
```

That is exactly what the 1D address formula does.

---

25. Key Terms

Term	Simple Meaning
Array	A collection of same-type values under one name
Heap array	An array created dynamically in heap memory
Pointer	A variable that stores a memory address
Contiguous memory	Memory placed side by side
Resizing	Creating a bigger array and copying old values into it
Helper pointer	A second pointer used temporarily during resizing
Memory leak	Heap memory that was not released
`malloc`	C function used to create heap memory
`free`	C function used to release heap memory
Base address	Starting address of the array
Index	Position number of an array element

Term	Simple Meaning
Word size	Number of bytes used by one element
0-indexing	Counting starts from 0
1-indexing	Counting starts from 1
Address formula	Formula used to calculate where an element is stored

---

26. Week 3 Formula Summary

0-indexed formula

Used in C and C++:

```
Address of A[i] = L0 + (i * w)
```

1-indexed formula

Used in some textbook examples or other languages:

```
Address of A[i] = L0 + ((i - 1) * w)
```

Meaning of symbols

```
L0 = base address
i  = index
w  = word size
```

---

27. Week 3 Summary

By the end of Week 3, you should understand that:

- Arrays need contiguous memory.
- Arrays cannot directly grow because the next memory locations may already be used.
- Heap arrays can be manually resized.
- Manual resizing means creating a bigger array, copying values, freeing the old array, and redirecting the pointer.

- `Q` is used as a helper pointer during resizing.
- Forgetting to free old memory causes a memory leak.
- Freeing memory too early is dangerous.
- New spaces in a resized array should be initialized.
- The computer calculates an array element address instead of searching for it.
- The base address is the starting address of the array.
- The word size is the number of bytes used by one element.
- In C and C++, arrays are 0-indexed.
- The 0-indexed formula is `Address of A[i] = L0 + (i * w)`.
- The 1-indexed formula is `Address of A[i] = L0 + ((i - 1) * w)`.
- 0-indexing makes address calculation simpler.

---

28. Week 3 Checkpoint Questions

Try answering these without looking at the notes.

1. Why can arrays not grow directly?
2. What does contiguous memory mean?
3. Why does resizing need a new bigger array?
4. What is the role of pointer `P`?
5. What is the role of helper pointer `Q`?
6. Why should we copy values from `P` to `Q` before freeing `P`?
7. What happens if we forget to call `free(P)`?
8. Why is `Q = NULL` used after `P = Q`?
9. What is a memory leak?
10. What is the base address of an array?
11. What is word size?
12. What is an index?
13. What is the 0-indexed address formula?
14. What is the 1-indexed address formula?
15. Why is `A[3]` the fourth element in C and C++?
16. Why does the address increase by 4 in an integer array?
17. Why does C/C++ use 0-based indexing?
18. What is the address of `A[4]` if `L0 = 300` and `w = 4`?
19. What is the address of `A[3]` in a 1-indexed array if `L0 = 500` and `w = 2`?
20. Why is array access considered fast?

---

29. Practice Tasks

Task 1: Resize from 5 to 8

Write a C program that:

1. Creates a heap array of size 5.
2. Stores values `10, 20, 30, 40, 50`.
3. Creates a bigger heap array of size 8.
4. Copies the old values.
5. Sets the new spaces to `0`.
6. Frees the old array.
7. Makes `P` point to the bigger array.
8. Prints all 8 values.
9. Frees the final array.

Expected final output:

```
10 20 30 40 50 0 0 0
```

Task 2: Manual Address Calculation

Calculate the address of `A[5]` using 0-indexing.

Given:

```
L0 = 1000
w = 4
i = 5
```

Formula:

```
Address of A[i] = L0 + (i * w)
```

Try solving it before checking the answer.

Answer:

```
Address of A[5] = 1000 + (5 * 4)
                = 1000 + 20
                = 1020
```

Task 3: Compare 0-Indexed and 1-Indexed

Given:

```
L0 = 400
w = 2
i = 4
```

Find the address using:

1. 0-indexing
2. 1-indexing

0-indexed answer:

```
Address of A[4] = 400 + (4 * 2)
                = 408
```

1-indexed answer:

```
Address of A[4] = 400 + ((4 - 1) * 2)
                = 400 + 6
                = 406
```

---

30. Simple Final Memory Aid

```
Array = same-type values under one name
Contiguous = side by side
Heap array = dynamic array
Pointer = stores address
Resize = create bigger block and copy
P = main pointer
Q = helper pointer
free(P) = release old memory
```

```
Q = NULL = helper is no longer used
Base address = address of first element
Word size = bytes per element
Index = position number
0-indexed formula = L0 + (i * w)
1-indexed formula = L0 + ((i - 1) * w)
```

Most important Week 3 idea:

```
The computer does not search for A[i].
It calculates the address of A[i].
```

That is why array access is fast.